

Distributed Cinema Reservation System with Apache Cassandra

Course: Big Data and Distributed Processing **Team:** Baris Surmelioglu and Ozan Polat

Description

This project is a distributed database application for cinema seat reservation management. The application uses Apache Cassandra as the distributed database, a Node.js/Express backend API, and a React graphical user interface. Users can create, update, view, cancel, and bulk cancel reservations. The project focuses on distributed data management, reservation consistency, and concurrent request handling.

Architecture and Technologies

Layer	Technology / Purpose
Database	Apache Cassandra cluster running with Docker Compose
Backend	Node.js, Express.js, Cassandra Driver
Frontend	React and Vite GUI
Testing	Node.js stress-test scripts using Axios

Main Functionalities

The system supports reservation creation, reservation update, reservation lookup, viewing who made each reservation, single reservation cancellation, and bulk cancellation of multiple reservations. The GUI works as a reservation management interface. Authentication and user authorization are outside the project scope because the goal of the assignment is distributed database functionality.

Cassandra Data Model

Cassandra tables are designed according to the queries required by the application. The system uses denormalized tables instead of relational joins: **reservations_by_session** for session seat maps, **reservations_by_id** for direct reservation lookup/update/cancellation, **reservations_by_customer** for customer-based lookup, and **reservation_events** for simple operation history.

The main reservation table uses the partition key (**movie_id, session_id**) and the clustering key **seat_number**. This allows the application to list reservations for a selected movie session and prevents duplicate seat records inside the same session.

Consistency and Double Booking Prevention

To prevent two clients from reserving the same seat at the same time, the backend uses Cassandra lightweight transactions with **IF NOT EXISTS** when inserting a reservation into **reservations_by_session**. If another client already reserved the same movie, session, and seat combination, the new request is rejected.

Stress Tests

The project includes five stress tests: fast repeated same request, random requests from multiple clients, two clients trying to occupy seats at the same time, repeated cancellation and reservation, and bulk cancellation. The tests confirmed that the system handles concurrent requests without double booking and that cancellation operations work correctly.

Problems Encountered

The main challenge was Cassandra data modeling, because Cassandra requires query-based table design instead of relational joins. Another challenge was handling concurrent requests for the same seat. This was solved with Cassandra lightweight transactions. The project also required validating seat numbers both in the GUI and backend to prevent invalid reservation updates.

Conclusion

The final application demonstrates a working distributed database reservation system. It satisfies the core requirements of the assignment: multi-node Cassandra usage, GUI application, reservation create/update/view/cancel operations, pair-project bulk cancellation, and stress testing for concurrent operations.